CineIQ Report

Purshotam Kumar and Uday Kumar

CineIQ: Hybrid Movie Recommendation Engine

Authors: Purshotam Kumar and Uday Kumar
Date: May 2026
Code: github.com/proust-19/CINEIQ

---

1. Abstract

Content discovery on modern streaming platforms is opaque, biased toward promoted titles, and traps users in recommendation loops. CineIQ addresses this by building an open, explainable hybrid movie recommendation engine that combines multiple machine learning strategies: content-based filtering, collaborative filtering via matrix factorization, and sentiment-aware re-ranking. The system is deployed as a FastAPI REST service with an interactive Streamlit dashboard, providing transparent and interpretable recommendations.

2. Problem Statement

Streaming platforms like Netflix, Amazon Prime, and Disney+ rely on proprietary recommendation algorithms that prioritize engagement metrics and promoted content over user satisfaction. Users often find themselves stuck in recommendation loops – being suggested the same type of content repeatedly without discovery of diverse or niche titles. Additionally, these black-box systems offer no explanation for why a particular title is recommended.

CineIQ tackles three core challenges:

1. **Cold-start recommendations** for new users who have no rating history
2. **Diverse discovery** – avoiding the filter bubble effect
3. **Explainability** – every recommendation surfaces a human-readable reason

3. Dataset

MovieLens 1M (grouplens.org/datasets/movielens/1m/) is a stable benchmark dataset containing:

Statistic	Value
Ratings	1,000,209
Users	6,040
Movies	3,883
Rating scale	1-5 (integer)
Time span	2000-2003
Genres	18 categories (Action, Comedy, Drama, etc.)

Each movie is annotated with one or more genre labels. The dataset does not include review text, so sentiment is proxied from rating values.

4. System Architecture

CineIQ follows a modular pipeline with four stages:

4.1 Content-Based Filtering (TF-IDF + Cosine Similarity)

Genre labels are cleaned (pipe-delimited genres converted to space-separated tokens) and vectorized using TF-IDF (Term Frequency-Inverse Document Frequency) with English stop-word removal. A 3883 x 3883 cosine similarity matrix captures pairwise genre affinity between all movies. Given a seed movie, the top-50 most similar candidates are retrieved.

4.2 Collaborative Filtering (SVD)

Using the Surprise library, an SVD (Singular Value Decomposition) model with 100 latent factors is trained over 20 epochs on the full user-item rating matrix. Three-fold cross-validation yields an RMSE of **0.8861**, which is state-of-the-art for the MovieLens 1M benchmark. The trained model predicts ratings for all unrated movies per user.

4.3 Weighted Ensemble

Individual recommenders are combined via a normalized weighted sum:

- **SVD score (50%)** – strongest signal from user behavior patterns
- **Content score (30%)** – ensures genre relevance to the seed movie
- **Popularity score (20%)** – global average rating as a fallback

Each component is min-max normalized to [0, 1] before combination to prevent any single component from dominating.

4.4 Sentiment Re-Ranking

Since MovieLens 1M lacks review text, a sentiment proxy is derived from rating values:

- Rating >= 4 -> Positive (+1.0)
- Rating = 3 -> Neutral (0.0)
- Rating <= 2 -> Negative (-1.0)

Per-movie aggregate sentiment is computed as the mean polarity weighted by log(review count) to account for confidence, then min-max normalized to [0, 1]. The final score combines the ensemble output with a 30% sentiment weight:

```
final_score = 0.7 * ensemble_score + 0.3 * sentiment_score
```

4.5 API & Dashboard

- **FastAPI** (`/recommend`, `/similar`) serves recommendations in JSON
- **Streamlit** dashboard provides an interactive UI with Plotly visualizations (score bar chart, genre distribution pie chart)
- The API is stateless and loads all pre-trained artifacts at startup for low-latency inference

5. Implementation

The system is implemented in Python with the following pipeline:

Step	Notebook	Output
1. EDA	`01_eda.ipynb`	Data statistics, visualizations, merged CSV
2. Content Model	`02_content_based.ipynb`	`cosine_sim.pkl`, `indices.pkl`

Step	Notebook	Output
3. SVD Model	`03_collaborative_svd.ipynb`	`svd_model.pkl`
4. Ensemble	`04_ensemble.ipynb`	`ensemble_weights.json`
5. Sentiment	`05_sentiment.ipynb`	`sentiment.pkl`, `movie_sentiment.csv`

Notebooks are designed to run sequentially; each saves artifacts consumed by the next stage and the API server.

6. Results & Evaluation

6.1 SVD Model Performance

3-fold cross-validation results:

Fold	RMSE	Fit Time (s)	Test Time (s)
1	0.8862	4.68	2.08
2	0.8859	4.52	1.62
3	0.8861	4.43	1.85
Mean	**0.8861**	**4.54**	**1.85**

6.2 Qualitative Results

Testing with user_id=1 and "Toy Story (1995)":

Rank	Movie	Score
1	Toy Story 2 (1999)	0.936
2	Wrong Trousers, The (1993)	0.922
3	Bug's Life, A (1998)	0.918
4	Chicken Run (2000)	0.912
5	Iron Giant, The (1999)	0.897

After sentiment re-ranking, movies with stronger audience reception are boosted in the rankings, providing a more nuanced final list.

7. Future Work / Upgradation

7.1 Scale to MovieLens 25M

Retrain all models on the 25M rating dataset (25x larger) to improve coverage for niche and long-tail movies. The SVD training complexity is O(n_factors x nnz), which scales linearly with the number of ratings.

7.2 Deep Learning Models

Replace SVD with Neural Collaborative Filtering (NCF) or two-tower models to capture non-linear user-item interactions. A multi-layer perceptron can learn deeper feature interactions beyond the linear factorization of SVD.

7.3 Review Text Sentiment with Transformers

Integrate actual IMDB review text using DistilBERT or RoBERTa for genuine sentiment signals instead of rating proxies. This would capture nuanced audience reactions (e.g., a 3-star movie with passionate reviews).

7.4 Real-Time Online Learning

Implement incremental model updates using algorithms like FTRL (Follow The Regularized Leader) or streaming SVD so recommendations evolve immediately with new ratings without requiring full retraining.

7.5 Cold-Start with Side Information

Use director, cast, release year, and plot embeddings (from Sentence-BERT) for content features. This enables recommending new or unrated movies by measuring similarity in a richer feature space.

7.6 A/B Testing Framework

Add an experimentation layer to compare ensemble variants, test different weights, and measure online engagement metrics like click-through rate and watch time.

8. Conclusion

CineIQ demonstrates a production-ready hybrid recommendation system combining collaborative filtering, content-based filtering, and sentiment analysis. The modular architecture allows independent improvement of each component, and the API-first design enables easy integration into front-end applications. With an SVD RMSE of 0.8861 and a weighted ensemble that balances user behavior, genre affinity, and audience sentiment, the system provides diverse, explainable, and personalized movie recommendations.

The project serves as both a functional recommendation engine and a reference architecture for hybrid ML systems – with clear pathways for scaling to larger datasets, adopting deep learning, and incorporating richer signals through the proposed upgradation roadmap.